Trackify Document Management System

Master Database Blueprint: Architecture, Integrity, Normalization, Data Flows & Defense Guide

Target DBMS: MySQL 8.0+ / MariaDB 10.4+	**Database Name:** `dms_db`	**Schema Version:** 2.0 Hardened	**Entity Count:** 16 Tables, 1 View

Executive Purpose: This technical guide provides defense candidates, software engineers, and database administrators with a comprehensive, panel-ready understanding of the **Trackify (dms_db)** schema. It covers relational integrity, ACID compliance, Third Normal Form (3NF) architecture, zero-loss audit logging, multi-stage approval logic, and 18 defense questions with complete model answers.

1. Schema Overview & Data Dictionary

The database is structured into four cohesive sub-systems: **Identity & Access Management (IAM)**, **Document Ingestion & Version Control**, **Workflow & Governance**, and **Audit Trail & Communication**. Every table enforces strict typing, declarative foreign key rules, and domain-level validation.

Table Name	Primary Key	Core Foreign Keys	Functional Purpose
departments	`id (INT AI)`	head_user_id → users(id)	Defines organizational units (HR, Finance, IT). Links department heads.
users	`id (INT AI)`	department_id → departments(id)	Stores credentials (bcrypt), role (admin, dept_head, staff), lockout, and soft-delete state.
categories	`id (INT AI)`	None	Classifies files (Memo, Contract, Policy). Validates hex color badge with CHECK.
tags	`id (INT AI)`	None	Atomic keyword lookup table eliminating comma-delimited strings (1NF compliance).
documents	`id (INT AI)`	category_id, department_id, uploaded_by, current_version_id, parent_document_id	Central document entity. Manages status, priority, confidentiality, hash, and soft-delete.
document_tags	`(document_id, tag_id)`	document_id → documents tag_id → tags	Associative junction table resolving Many-to-Many (M:N) relationship between documents and tags.
document_versions	`id (INT AI)`	document_id → documents uploaded_by → users	Immutable revision history. Unique (document_id, version) ensures strict sequential revisions.
document_permissions	`id (INT AI)`	document_id, user_id, department_id, granted_by	Granular per-document Access Control List (ACL) granting read/comment/edit overrides beyond RBAC.
approval_workflows	`id (INT AI)`	document_id → documents approver_id → users	Multi-stage approval pipeline. Supports approval 'rounds' for resubmission tracking.
action_types	`id (TINYINT AI)`	None	Lookup dictionary of all auditable events (upload, approve, reject, login, etc.).
activity_logs	`id (INT AI)`	user_id → users document_id → documents action_type_id → action_types	Append-only tamper-proof audit trail. Preserves user_name_snapshot when users are removed.
login_attempts	`id (INT AI)`	None (Email indexed)	Rate limiting and brute-force prevention telemetry. Purged on a 24-hour cycle.

notifications	`id (INT AI)`	user_id → users document_id → documents	In-app notifications (approvals, mentions, deadlines) with read/unread tracking.
comments	`id (INT AI)`	document_id → documents user_id → users	Document discussion thread. SET NULL on user deletion prevents dangling or missing remarks.
calendar_tasks	`id (INT AI)`	department_id → departments created_by → users	Task scheduler. CHECK constraint ensures visibility='department' requires a valid department_id.
document_reference_seq	`id (INT AI)`	None (Unique prefix+year)	Atomic counter sequence generator preventing race conditions during reference number generation.
active_documents (View)	`N/A (Virtual Table)`	Points to documents	SQL View filtering out deleted_at IS NOT NULL and status='archived' for fast query safety.

2. Relational Mapping & Multiplicity (Cardinality)

Multiplicity defines the numerical relationship between occurrences of entities. The Trackify database implements four fundamental multiplicity patterns:

Relationship	Multiplicity	Foreign Key Location	Business Rule & Delete Policy
Department → Users	1 : N (One-to-Many)	users.department_id	A department has multiple employees. If a department is deleted, users are SET NULL (employees are preserved).
Department → Head User	1 : 1 (One-to-One Optional)	departments.head_user_id	A department may have at most one official Head. Circular reference resolved by creating users before altering departments.
User → Uploaded Documents	1 : N (One-to-Many)	documents.uploaded_by	A user can author many documents. Deleting an active author is RESTRICTed to prevent orphaned corporate files.
Document → Versions	1 : N (One-to-Many)	document_versions.document_id	One document has an audit trail of revisions. Deleting a document CASCADE deletes its revision logs.
Document → Current Version	1 : 1 (One-to-One Direct)	documents.current_version_id	Explicit pointer to the active production revision. Enables instant metadata retrieval without subqueries.
Document → Tags	M : N (Many-to-Many)	document_tags(document_id, tag_id)	A document has many tags; a tag labels many documents. Resolved via composite primary key associative entity.
Document → Approvers	1 : N (One-to-Many per Round)	approval_workflows.document_id	Sequenced pipeline of review steps. Grouped by 'round' to cleanly separate re-submissions after a rejection.
Document → Clones / Forks	1 : N (Self-Referential / Recursive)	documents.parent_document_id	A document can be duplicated or branched into a child document. If parent is deleted, child is SET NULL.

3. Normalization Analysis (1NF, 2NF, 3NF & Strategic Denormalization)

Database normalization eliminates redundant data, prevents insertion/update/deletion anomalies, and ensures data dependency integrity.

- **First Normal Form (1NF) — Atomicity & No Repeating Groups:** In the legacy schema, tags were stored as a comma-separated string (e.g. `'memo,urgent,hr'`) in `documents.tags`. This violates 1NF because individual values cannot be queried, indexed, or foreign-key enforced directly without expensive string pattern matching. The hardened schema achieves 1NF by creating the atomic `tags` table and the `document_tags` junction table.
- **Second Normal Form (2NF) — Full Functional Dependency:** 2NF requires 1NF and states that non-key attributes must depend on the *entire* candidate key, not just a subset. In `document_tags`, the primary key is composite: `(document_id, tag_id)`. There are no non-key columns in this table; every attribute constitutes the whole key. In `document_versions`, all columns (file_path, file_size, notes) depend completely on the synthetic primary key `id` (or the alternate composite candidate key `(document_id, version)`).
- **Third Normal Form (3NF) — Non-Transitive Dependency:** 3NF dictates that non-key columns must depend *only on the primary key* (no column can depend on another non-key column: 'the key, the whole key, and nothing but the key'). For example, document metadata does not store department head name or category color inside `documents`; it references `department_id` and `category_id`, fetching parent details via JOIN.
- **Strategic Controlled Denormalization (The Audit Exception):** Pure 3NF is intentionally relaxed in one critical scenario: `activity_logs.user_name_snapshot`. In standard 3NF, the actor's name should be queried from `users.id`. However, in an enterprise compliance environment, if a user account is deleted or legally purged, foreign key constraints would force the audit log to SET NULL. By capturing `user_name_snapshot` at the time of the event, the audit trail remains permanently readable and legally immutable even if the original user record vanishes.

4. Data Integrity & Defensive Constraints

Integrity constraints prevent invalid, corrupt, or inconsistent data from entering the database regardless of bugs in the front-end application layer.

Integrity Type	Mechanism in Schema	Defensive Purpose & Edge Case Prevented
Entity Integrity	PRIMARY KEY + AUTO_INCREMENT on all tables	Guarantees that every tuple has a unique, non-null, immutable row identifier.
Domain Integrity	ENUM('admin','dept_head','staff') CHECK (color REGEXP '^#[0-9a-fA-F]{6}$') BIGINT UNSIGNED file_size	Restricts column inputs to legal values. Hex color check stops invalid CSS. BIGINT ensures file size tracking > 2GB without overflow.
State Coherence Integrity	CHECK ((status='pending' AND acted_at IS NULL) OR (status<>'pending' AND acted_at IS NOT NULL))	Enforced on `approval_workflows`. Prevents invalid review states where an item is marked approved without a timestamp, or vice-versa.
Business Logic Integrity	CHECK (visibility='private' OR department_id IS NOT NULL)	Enforced on `calendar_tasks`. Prevents orphan department events where visibility is 'department' but no department was assigned.
Referential Integrity	FOREIGN KEY ... ON DELETE RESTRICT (users on documents) ON DELETE SET NULL (logs/comments) ON DELETE CASCADE (versions/tags)	Prevents orphaned rows. Prevents accidental destruction of company documents when removing users. Keeps audit logs even if users depart.
Concurrency & Deduplication	UNIQUE(document_id, version) UNIQUE(document_id, round, step_order) file_hash CHAR(64)	Prevents duplicate version numbers during simultaneous uploads. Prevents duplicate approval steps in a round. Detects identical binary file uploads via SHA-256.
Temporal Longevity	DATETIME instead of TIMESTAMP	TIMESTAMP in MySQL is signed 32-bit integer ending on January 19, 2038 (Y2038 bug). DATETIME supports dates from year 1000 to 9999.

5. End-to-End Data Flows & Lifecycles

A. Document Upload & Ingestion Flow

1. User initiates upload. Application calculates SHA-256 hash of the binary file and queries `documents WHERE file_hash = ?` to check for duplicates.
2. Inside an ACID Transaction: A new row is inserted into `documents` with status='draft', version=1, file_hash, and uploaded_by.
3. Initial version row is inserted into `document_versions` (document_id, version=1, file_path, uploaded_by).
4. `documents.current_version_id` is updated to point to the created version record.
5. Tag tokens are split: existing tags are matched or inserted into `tags`, and pairs are bound in `document_tags`.
6. An entry is appended to `activity_logs` with action_type='upload', capturing actor IP and snapshot name. Transaction commits.

B. Multi-Stage Approval & Resubmission Flow

1. Author clicks 'Submit for Review'. Document status transitions from 'draft' to 'pending_review'.
2. The workflow engine looks up approval steps and inserts rows into `approval_workflows` with round=1, step_order=1..N, status='pending'.
3. Step 1 approver receives an in-app alert via `notifications` (type='approval_request').
4. Approver reviews and acts: If Approved, status becomes 'approved', acted_at=NOW(). If step 2 exists, step 2 becomes active. If final step passes, `documents.status` transitions to 'approved', and a reference number is generated via `document_reference_seq`.
5. Rejection Branch: If an approver clicks 'Reject', their row gets status='rejected', remarks are saved, and `documents.status` becomes 'rejected'.
6. Resubmission Round: When author modifies file and resubmits, the workflow queries `MAX(round) + 1`. New rows are inserted with round=2. The history of round 1 remains intact, preventing reviews from colliding or being overwritten.

C. Soft-Delete, Archival & Access Control Flow

1. Archival: Status becomes 'archived'. Document remains in storage for legal retention but is filtered out of regular active listings.
2. Soft Delete: When a user deletes a file, `documents.deleted_at = NOW()` and `deleted_by = user_id` are set. No rows are removed from disk.
3. Application Querying: Application queries the `active_documents` SQL View for operational lists, guaranteeing soft-deleted and archived files are automatically excluded without requiring manual WHERE clauses in every query.
4. Access Verification: System checks if user is Admin (universal access), belongs to the document's department, or possesses a specific grant in `document_permissions` matching confidentiality classification.

6. Defense Panelist Q&A; (Situational to Complex Technical)

Below are 18 rigorous defense questions simulating an academic capstone or technical industry panel, structured across three difficulty tiers.

TIER 1: CONCEPTUAL & FOUNDATIONAL

Q1: Why did you separate documents and document_versions into two different tables instead of having a single documents table with a version column?

Answer: Storing revisions in a single table causes massive data redundancy and violates 2NF. Document-level metadata (such as category, department, permissions, creation date, and approval history) does not change every time a typo is corrected in a file. By splitting into a 1:N relationship, `documents` retains stable parent identity, while `document_versions` acts as an append-only ledger of physical files, sizes, hashes, and change notes. This also allows rollback to previous files without altering foreign key links in other tables.

Q2: Why is the relationship between documents and tags Many-to-Many (M:N), and why did you create a junction table instead of storing comma-separated strings?

Answer: A document can have multiple topical tags ('Finance', 'Confidential', 'Audit'), and a single tag applies to thousands of documents. Storing them as a comma-separated string in `documents.tags` violates First Normal Form (1NF) because values are non-atomic. It forces inefficient `LIKE '%tag%'` table scans that bypass indexes and make tag renaming or referential integrity impossible. The associative table `document_tags(document_id, tag_id)` uses a composite primary key, enabling index seeks, O(1) tag lookup, and instant cascading cleanup.

Q3: What is the difference between your soft-delete mechanism and hard deletion, and why is hard deletion dangerous in this system?

Answer: Hard deletion physically executes `DELETE FROM documents`, permanently purging the row and triggering cascading deletes on child versions and comments. In an enterprise DMS, this destroys audit trails, breaks legal compliance, and makes accidental loss unrecoverable. Our soft-delete sets `deleted_at = DATETIME` and `deleted_by = user_id`. The data remains intact, can be restored instantly, and is safely hidden from active operational workflows using our `active_documents` view.

Q4: Why does the categories table have a color column with a CHECK constraint?

Answer: The UI renders visual category badges. Without schema-level constraints, a malformed string or XSS vector could be injected into `categories.color`. We enforce domain integrity using `CHECK (color REGEXP '^#[0-9a-fA-F]{6}$')`, guaranteeing that only valid 6-character hexadecimal color codes (e.g. #3B82F6) can ever be stored, shielding the frontend renderer from injection or style breakage.

Q5: Why did you use DATETIME instead of TIMESTAMP across all tables in your hardened schema?

Answer: MySQL's native `TIMESTAMP` datatype is stored as a 32-bit integer representing seconds since the Unix epoch (1970-01-01). On January 19, 2038, 32-bit timestamps will overflow (the Year 2038 problem), causing dates to flip to 1901 and crashing system logic. `DATETIME` uses an 8-byte/5-byte packed format storing values up to year 9999, completely immunizing the document archive against future timestamp overflow.

TIER 2: SITUATIONAL & BUSINESS LOGIC

Q6: What happens if an employee who authored 500 documents resigns and their user account is deleted?

Answer: In our hardened schema, the foreign key on `documents.uploaded_by` is defined as `ON DELETE RESTRICT`. The database engine will literally refuse to drop the user record with an integrity violation error. Instead of hard-deleting staff, our system enforces soft-deactivation: `users.is_active = 0` and `users.deleted_at = NOW()`. This ensures that all historical documents, authorship logs, and accountability records remain 100% intact and linked.

Q7: A document is rejected by an approver, revised by the author, and resubmitted. How does your schema prevent old approval comments from mixing with new ones?

Answer: In naive schemas, re-submitting overwrites the existing workflow rows or appends rows with duplicate step numbers, causing confusion. We solved this by adding an explicit `round INT NOT NULL DEFAULT 1` column and a unique constraint: `UNIQUE(document_id, round, step_order)`. When a document is rejected in Round 1 and resubmitted, new workflow steps are created with `round = 2`. Round 1 remains frozen for historical audit, while Round 2 tracks the live resubmission pipeline.

Q8: How does your database prevent a user from approving their own document (self-approval)?

Answer: Self-approval violates separation of duties. While the application service layer validates that `approver_id != documents.uploaded_by` prior to inserting workflow rows, the database enforces status integrity using: `CHECK ((status='pending' AND acted_at IS NULL) OR (status<>'pending' AND acted_at IS NOT NULL))`. Additionally, an approver cannot exist as an active approver on an unauthorized step due to step-order constraints.

Q9: What prevents two users from uploading Revision 2 of the exact same document at the exact same millisecond?

Answer: We enforce concurrency control at the storage engine level with the constraint: `UNIQUE KEY uq_doc_version (document_id, version)` on `document_versions`. If two concurrent HTTP requests attempt to write version 2 for document ID 45 simultaneously, MySQL's InnoDB row-level locking ensures the first transaction writes successfully while the second immediately fails with an integrity violation error (1062 Duplicate entry), completely preventing duplicate revision numbers.

Q10: Why do you have both role-based access control (RBAC) in users and an explicit document_permissions table?

Answer: RBAC (`admin`, `department_head`, `staff`) handles broad organizational governance: department heads see department documents, staff see their own work. However, enterprise workflows have cross-functional exceptions: an HR policy might need to be reviewed by a specific Finance auditor, or a highly sensitive executive contract must be hidden from normal department staff. `document_permissions` provides a granular Access Control List (ACL) supporting read/comment/edit overrides for specific users or external departments without elevating their overall system role.

Q11: Suppose two users approve two documents at the exact same moment. How do you prevent duplicate document reference numbers (e.g., REF-2026-0001)?

Answer: Naive systems execute `SELECT MAX(id) FROM documents`, which suffers from race conditions where two threads read the same max value and generate duplicate numbers. We created the dedicated atomic counter table `document_reference_seq (prefix, year, next_val)`. Inside a database transaction, we execute an atomic update: `UPDATE document_reference_seq SET next_val = LAST_INSERT_ID(next_val + 1) WHERE prefix = ? AND year = ?`. This utilizes InnoDB's internal sequence locking to guarantee that each transaction receives a strictly sequential, non-colliding integer.

TIER 3: COMPLEX TECHNICAL & DEFENSE INQUISITION

Q12: In activity_logs, you store user_name_snapshot which duplicates the user's name already in users. Doesn't this violate Third Normal Form (3NF)? How do you defend this?

Answer: Yes, strictly speaking, it violates 3NF by introducing transitive redundancy. However, this is an intentional, industry-standard pattern called **Audit Durability Denormalization**. In an enterprise audit log, immutability takes precedence over pure normalization. If an employee changes their name, or if their account is anonymized under privacy laws (like GDPR) or purged, querying an audit trail using an inner join on `users.id` would either return NULL or show the altered name retrospectively. Storing a snapshot guarantees an unalterable historical record of who executed the transaction at that exact moment in time.

Q13: Notice that documents has both parent_document_id and current_version_id, while document_versions points back to documents. Isn't this circular and redundant?

Answer: They serve two completely distinct architectural purposes: (1) `parent_document_id` is a self-referencing relationship representing *cloning or branching* (e.g. 'Document B was created as a copy of Template A'). It is NOT used for revisioning. (2) `current_version_id` is a 1:1 optimization pointer referencing the active production record in `document_versions`. Without this pointer, every single query searching for a document's current file path, size, or hash would require an expensive correlated subquery (`WHERE version = (SELECT MAX(version)...)`). Having an explicit pointer allows instant O(1) joins while preserving the 1:N revision history.

Q14: Explain the circular dependency between departments and users. How did you resolve it during schema generation?

Answer: `users` references `departments(id)` to assign employees to an organization, while `departments` references `users(id)` via `head_user_id` to designate who manages that department. If both foreign keys were declared in their initial `CREATE TABLE` statements, neither table could be created first without throwing a foreign key constraint failure. We resolved this cleanly by creating `departments` with `head_user_id INT NULL` first, then creating `users` with its FK to `departments`, and finally executing an `ALTER TABLE departments ADD CONSTRAINT fk_dept_head FOREIGN KEY...`. This breaks the cycle cleanly during migrations.

Q15: How does your database handle ACID properties during a file upload?

Answer: A file upload spans multiple entities (`documents`, `document_versions`, `document_tags`, and `activity_logs`). We wrap this workflow in a single database transaction: (1) **Atomicity:** If tag insertion fails or the version pointer fails, the entire transaction rolls back; no orphaned document record is left behind. (2) **Consistency:** Foreign keys ensure no version exists without a valid document or user. (3) **Isolation:** Under InnoDB's default REPEATABLE READ isolation level, concurrent transactions cannot read partial uncommitted version rows. (4) **Durability:** Once committed, the InnoDB redo log ensures writes survive power failures or crashes.

Q16: Why did you add file_hash CHAR(64) to both documents and document_versions?

Answer: `file_hash` stores the cryptographic SHA-256 hash (64 hexadecimal characters) of the file payload. It fulfills two enterprise needs: (1) **Data Deduplication:** If a user attempts to upload a 50MB PDF that already exists in the system under a different filename, the system compares hashes and reuses physical storage instead of duplicating disk space. (2) **Integrity Verification:** It prevents silent file corruption (bit rot) or unauthorized direct filesystem tampering. Periodically or upon download, the file hash can be re-computed and compared to the database hash.

Q17: What indexing strategy did you employ in this database, and how do you prevent full table scans when millions of documents exist?

Answer: We implemented targeted composite and covering indexes based on real query access patterns: (1) `idx_documents_dept_stat(department_id, status)` satisfies dashboard filtering without table scans. (2) `idx_notif_user_read(user_id, is_read)` powers the high-frequency unread notification badge query instantly. (3) `idx_al_doc_created(document_id, created_at)` renders audit logs in reverse chronological order via index scan. (4) `FULLTEXT INDEX idx_documents_ft(title, description)` allows tokenized full-text keyword searches using boolean mode instead of disastrous `LIKE '%keyword%'` scans that cripple production engines.

Q18: If your system scales to 10 million documents and 50 million versions, what database architecture or partitioning would you recommend next?

Answer: To scale beyond single-instance InnoDB limits: (1) **Horizontal Table Partitioning:** Partition `activity_logs` and `login_attempts` by RANGE on `created_at` (e.g. yearly or monthly partitions) so old historical partitions can be archived or dropped with zero table lock. (2) **Read-Replica Architecture:** Route write transactions (`INSERT/UPDATE`) to an active Master database and delegate heavy analytical queries and search reads to read-replicas. (3) **Blob Offloading:** Ensure physical document binaries reside in Object Storage (e.g. MinIO or AWS S3), keeping the MySQL database lean and storing only metadata, hashes, and storage paths.